¿Dónde viajarías?

Código del proyecto

Aplicación web hecha con Python (Flask) que recomienda países según el tipo de viaje, el clima y el presupuesto. Consulta la API REST Countries v5, permite buscar países y descubrir un destino aleatorio. Las recomendaciones se basan en reglas programadas, sin inteligencia artificial.

Alumnos: Uñates Ivan- Ayala Rosalinda

app.py

```
import json
import os
import random
import time
import unicodedata
from pathlib import Path

import requests
from flask import Flask, jsonify, render_template, request
from werkzeug.exceptions import HTTPException

from datos import DESTINOS

API = "https://api.restcountries.com/countries/v5"
CAMPOS =
"names,codes.alpha_2,codes.alpha_3,capitals,region,subregion,population,flag,languages,currencies"
NIVELES = ["bajo", "medio", "alto"]
TTL = 24 * 3600  # los datos de países casi no cambian: cuidamos tu cuota de la API
ARCHIVO_CACHE = Path(__file__).with_name("cache_paises.json")

app = Flask(__name__)
_cache = {"t": 0, "paises": []}

class ApiError(Exception):
    """Problema al consultar REST Countries (clave faltante, cuota, etc.)."""

def normalizar(texto):
    """Minúsculas y sin tildes, para buscar sin importar acentos."""
    s = unicodedata.normalize("NFD", texto.lower())
    return "".join(c for c in s if unicodedata.category(c) != "Mn")

def leer_clave():
    """La clave puede estar en la variable de entorno o en el archivo .env."""
    clave = os.environ.get("RESTCOUNTRIES_API_KEY", "").strip()
    env = Path(__file__).with_name(".env")
    if not clave and env.exists():
        for linea in env.read_text(encoding="utf-8-sig").splitlines():
            k, _, v = linea.partition("=")
            if k.strip() == "RESTCOUNTRIES_API_KEY":
                clave = v.strip().strip("\"'")
    return clave

_CODIGO = ("iso", "bcp", "code", "alpha", "tag", "numeric", "id", "symbol", "native")

def _hojas(x, ruta=""):
```

```
    """Todos los textos de una estructura anidada, junto con su ruta (ej. 'name.english')."""
    if isinstance(x, str):
        yield ruta.lower(), x
    elif isinstance(x, dict):
        for k, v in x.items():
            yield from _hojas(v, f"{ruta}.{k}" if ruta else str(k))
    elif isinstance(x, list):
        for v in x:
            yield from _hojas(v, ruta)

def _elegir(x, claves):
    """Elige el texto más parecido a un nombre, sin importar cómo venga anidado."""
    hojas = list(_hojas(x))
    for clave in claves:
        for ruta, t in hojas:
            if clave in ruta and t.strip():
                return t
    for ruta, t in hojas:
        if not any(e in ruta for e in _CODIGO) and len(t) > 3:
            return t
    return ""

def _idiomas(l):
    items = list(l.values()) if isinstance(l, dict) else (l or [])
    nombres = [_elegir(i, ["english", "name"]) for i in items]
    return list(dict.fromkeys(n for n in nombres if n))

def _monedas(c):
    if isinstance(c, dict) and c and all(len(str(k)) == 3 and str(k).isalpha() for k in c):
        items = list(c.items())  # formato {"ARS": {...}}
    elif isinstance(c, dict):
        items = [("", c)]
    else:
        items = [("", v) for v in (c or [])]
    salida = []
    for codigo, v in items:
        hojas = list(_hojas(v))
        if isinstance(v, str):
            nombre, codigo = "", codigo or v
        else:
            nombre = _elegir(v, ["english", "name"])
            codigo = codigo or next((t for r, t in hojas if any(k in r for k in ("code", "iso",
"alpha"))), "")
        simbolo = next((t for r, t in hojas if "symbol" in r), "")
        extra = ", ".join(x for x in (codigo, simbolo if simbolo != codigo else "") if x)
        texto = f"{nombre} ({extra})" if nombre and extra else (nombre or extra or next((t for _, t in
hojas), ""))
        if texto:
            salida.append(texto)
    return list(dict.fromkeys(salida))

def formatear(o):
    """Convierte un país de REST Countries v5 al formato que usa nuestra web."""
    nombres = o.get("names") or {}
    spa = (nombres.get("translations") or {}).get("spa")
    spa = spa if isinstance(spa, dict) else {}
    caps = [c.get("name") if isinstance(c, dict) else c for c in o.get("capitals") or []]
    codigos = o.get("codes") or {}
    flag = o.get("flag") or {}
    a2 = (codigos.get("alpha_2") or "").lower()
    bandera = flag.get("url_svg") or flag.get("url_png") or (
        f"https://flags.restcountries.com/v5/w320/{a2}.png" if a2 else "")
    return {
        "codigo": codigos.get("alpha_3", ""),
        "nombre": spa.get("common") or nombres.get("common", ""),
```

```
        "nombre_en": nombres.get("common", ""),
        "capital": ", ".join(c for c in caps if c) or "Sin capital oficial",
        "region": o.get("subregion") or o.get("region") or "",
        "poblacion": o.get("population") or 0,
        "bandera": bandera,
        "idiomas": _idiomas(o.get("languages")),
        "monedas": _monedas(o.get("currencies")),
    }

def descargar():
    """Pide los ~250 países a la API v5, de a 100 por página (límite del plan gratis)."""
    clave = leer_clave()
    if not clave:
        raise ApiError("Falta la API key de REST Countries. Creala gratis y guardala en el archivo .env
(ver README).")
    paises, offset = [], 0
    for _ in range(10):
        r = requests.get(API, headers={"Authorization": f"Bearer {clave}"}, timeout=20,
                         params={"limit": 100, "offset": offset, "response_fields": CAMPOS})
        try:
            cuerpo = r.json()
        except ValueError:
            cuerpo = {}
        if r.status_code != 200 or not isinstance(cuerpo, dict) or "data" not in cuerpo:
            errores = cuerpo.get("errors") if isinstance(cuerpo, dict) else None
            msg = errores[0].get("message") if errores else f"HTTP {r.status_code}"
            raise ApiError(f"REST Countries respondió: {msg}")
        datos = cuerpo["data"]
        paises += [formatear(o) for o in datos.get("objects", [])]
        if not (datos.get("meta") or {}).get("more"):
            break
        offset += 100
    return paises

def _con_info(paises):
    for p in paises:
        p["info"] = DESTINOS.get(p["codigo"])
    return paises

def cargar_paises():
    """Devuelve los países desde memoria, desde el archivo de caché o desde la API."""
    ahora = time.time()
    if _cache["paises"] and ahora - _cache["t"] < TTL:
        return _cache["paises"]
    if not _cache["paises"] and ARCHIVO_CACHE.exists():
        try:
            guardado = json.loads(ARCHIVO_CACHE.read_text(encoding="utf-8"))
            if guardado.get("v") == 2 and ahora - guardado["t"] < TTL and guardado["paises"]:
                _cache.update(t=guardado["t"], paises=_con_info(guardado["paises"]))
                return _cache["paises"]
        except (ValueError, KeyError, OSError):
            pass
    try:
        paises = descargar()
    except (requests.RequestException, ApiError):
        if _cache["paises"]:  # si falla la API, usamos la copia anterior
            return _cache["paises"]
        raise
    if not paises:
        raise ApiError("REST Countries no devolvió países. Revisá tu API key.")
    try:
        ARCHIVO_CACHE.write_text(json.dumps({"v": 2, "t": ahora, "paises": paises}), encoding="utf-8")
    except OSError:
        pass
    _cache.update(t=ahora, paises=_con_info(paises))
```

```
    return _cache["paises"]

@app.errorhandler(ApiError)
def error_clave(e):
    return jsonify(error=str(e)), 503

@app.errorhandler(requests.RequestException)
def error_api(_):
    return jsonify(error="No pudimos conectar con REST Countries."), 503

@app.errorhandler(Exception)
def error_general(e):
    """Las rutas /api siempre responden JSON, nunca una página HTML de error."""
    if not request.path.startswith("/api/"):
        if isinstance(e, HTTPException):
            return e
        raise e
    app.logger.exception("Error en %s", request.path)
    codigo = e.code if isinstance(e, HTTPException) else 500
    return jsonify(error=f"Error del servidor: {e}"), codigo

@app.get("/")
def inicio():
    return render_template("index.html")

@app.get("/api/recomendar")
def recomendar():
    tipo = request.args.get("tipo", "cualquiera")
    clima = request.args.get("clima", "cualquiera")
    pres = request.args.get("presupuesto", "cualquiera")
    resultados = []
    for p in cargar_paises():
        d = p["info"]
        if not d:
            continue
        puntos, razones = 0, []
        if tipo == "cualquiera" or tipo in d["tipos"]:
            puntos += 40
            if tipo != "cualquiera":
                razones.append(f"Ideal para {tipo}")
        if clima == "cualquiera" or clima in d["clima"]:
            puntos += 35
            if clima != "cualquiera":
                razones.append(f"Clima {clima}")
        if pres == "cualquiera":
            puntos += 25
        else:
            dif = abs(NIVELES.index(pres) - NIVELES.index(d["presupuesto"]))
            puntos += {0: 25, 1: 10}.get(dif, 0)
            if dif == 0:
                razones.append(f"Presupuesto {pres}")
        if puntos >= 65:  # al menos dos criterios en común
            resultados.append({**p, "puntaje": puntos, "razones": razones})
    random.shuffle(resultados)  # variedad entre destinos con igual puntaje
    resultados.sort(key=lambda x: -x["puntaje"])
    return jsonify(paises=resultados[:9])

@app.get("/api/buscar")
def buscar():
    q = normalizar(request.args.get("q", "").strip())
    if len(q) < 2:
        return jsonify(paises=[])
    hallados = [p for p in cargar_paises()
                if q in normalizar(p["nombre"]) or q in normalizar(p["nombre_en"])
                or q in normalizar(p["capital"])]
```

```
    hallados.sort(key=lambda p: p["nombre"])
    return jsonify(paises=hallados[:12])

@app.get("/api/debug")
def debug():
    """Muestra cómo manda la API los idiomas y monedas (útil si algo sale vacío)."""
    r = requests.get(API, headers={"Authorization": f"Bearer {leer_clave()}"}, timeout=20,
                     params={"limit": 1, "response_fields": "names.common,languages,currencies"})
    return jsonify(r.json())

@app.get("/api/aleatorio")
def aleatorio():
    return jsonify(paises=[random.choice(cargar_paises())])

if __name__ == "__main__":
    app.run(debug=True)
```

datos.py

Datos complementarios: clima, tipo de viaje y presupuesto por país

```
1  """Datos complementarios (curados a mano) para las recomendaciones.
2  REST Countries no informa clima, tipo de viaje ni costo, así que se definen acá. 3 Para
   sumar un país, agregá una línea con su código ISO de 3 letras."""  4   5  def d(clima,
   tipos, presupuesto, nota):
  6       return {"clima": clima, "tipos": tipos, "presupuesto": presupuesto, "nota": nota}
7
8       DESTINOS = {
9       "ARG": d(["templado", "frio"], ["naturaleza", "aventura", "gastronomia", "ciudad"], "medio",
        »  "Glaciares, vinos y asado de punta a punta."),
10      "BRA": d(["tropical"], ["playa", "naturaleza", "ciudad"], "medio", "Playas infinitas y la se
        » lva amazónica."),
11      "CHL": d(["templado", "seco", "frio"], ["aventura", "naturaleza", "gastronomia"], "medio", "
        » Del desierto de Atacama a la Patagonia."),
12      "PER": d(["seco", "templado", "tropical"], ["cultura", "aventura", "gastronomia"], "bajo", "
        » Machu Picchu y una cocina de fama mundial."),
13      "COL": d(["tropical", "templado"], ["naturaleza", "playa", "ciudad", "gastronomia"], "bajo",
        »  "Caribe, café y ciudades llenas de color."),
14      "MEX": d(["tropical", "seco"], ["playa", "cultura", "gastronomia"], "medio", "Ruinas mayas,
        » cenotes y tacos."),
15      "CRI": d(["tropical"], ["naturaleza", "playa", "aventura"], "medio", "Volcanes, selva y vida
        »  silvestre."),  16      "CUB": d(["tropical"], ["playa", "cultura"], "bajo", "Música, autos
        clásicos y mar turquesa.
     » "),
17      "URY": d(["templado"], ["playa", "ciudad", "gastronomia"], "medio", "Punta del Este y Coloni
        » a del Sacramento."),
18      "BOL": d(["seco", "frio"], ["aventura", "cultura", "naturaleza"], "bajo", "El salar de Uyuni
        »  y paisajes de altura."),
19      "ECU": d(["tropical", "templado"], ["naturaleza", "aventura"], "bajo", "Galápagos, Andes y A
        » mazonia."),
20      "ESP": d(["templado", "seco"], ["cultura", "playa", "gastronomia", "ciudad"], "medio", "Tapa
        » s, arquitectura y costas soleadas."),
21      "ITA": d(["templado"], ["cultura", "gastronomia", "ciudad"], "medio", "Arte, historia y past
        » a en cada esquina."),
22      "FRA": d(["templado"], ["cultura", "gastronomia", "ciudad"], "alto", "París, castillos y viñ
        » edos."),
23      "PRT": d(["templado"], ["playa", "gastronomia", "cultura"], "medio", "Lisboa, Oporto y el Al
        » garve."),
24      "GRC": d(["templado", "seco"], ["playa", "cultura"], "medio", "Islas blancas y ruinas milena
        » rias."),
25      "DEU": d(["templado"], ["ciudad", "cultura"], "medio", "Historia, cerveza y mercados navideñ
        » os."),
26      "GBR": d(["templado"], ["ciudad", "cultura"], "alto", "Londres, castillos y campiña verde.")
     » ,
27      "ISL": d(["frio"], ["naturaleza", "aventura"], "alto", "Auroras, geiseres y cascadas."),
28      "NOR": d(["frio"], ["naturaleza", "aventura"], "alto", "Fiordos y sol de medianoche."), 29
```

```
        "CHE": d(["frio", "templado"], ["naturaleza", "aventura", "ciudad"], "alto", "Alpes, lagos y
»  trenes panorámicos."),
30      "HRV": d(["templado"], ["playa", "cultura"], "medio", "Costa adriática y murallas medievales
        » ."),
31      "TUR": d(["templado", "seco"], ["cultura", "gastronomia", "playa"], "bajo", "Estambul, Capad
     » ocia y la costa turquesa."),
32      "EGY": d(["seco"], ["cultura", "aventura"], "bajo", "Pirámides, templos y el río Nilo."), 33
        "MAR": d(["seco", "templado"], ["cultura", "gastronomia", "aventura"], "bajo", "Zocos, desie
» rto y ciudades de colores."),
34      "ZAF": d(["templado"], ["naturaleza", "aventura", "playa"], "medio", "Safaris, viñedos y Ciu
     » dad del Cabo."),
35      "KEN": d(["tropical"], ["naturaleza", "aventura"], "medio", "Safari y la gran migración."),
        36      "JPN": d(["templado"], ["cultura", "gastronomia", "ciudad"], "alto", "Templos,
        tecnología y
     » sushi."),
 37      "THA": d(["tropical"], ["playa", "gastronomia", "cultura"], "bajo", "Islas paradisíacas y
co
     » mida callejera."),  38      "VNM": d(["tropical"], ["gastronomia", "naturaleza", "cultura"],
"bajo", "Bahía de Ha Long y
     »  ciudades vibrantes."),
 39      "IND": d(["tropical", "seco"], ["cultura", "gastronomia", "aventura"], "bajo", "Palacios,
es
     » pecias y el Taj Mahal."),  40      "IDN": d(["tropical"], ["playa", "naturaleza",
"aventura"], "bajo", "Bali, volcanes y arroza
     » les."),
41      "ARE": d(["seco"], ["ciudad", "playa"], "alto", "Rascacielos, desierto y lujo."),
42      "AUS": d(["templado", "tropical", "seco"], ["playa", "naturaleza", "aventura", "ciudad"], "a
        » lto", "Gran Barrera de Coral y el outback."),
43      "NZL": d(["templado", "frio"], ["naturaleza", "aventura"], "alto", "Paisajes de película y d
        » eportes extremos."),
44      "CAN": d(["frio"], ["naturaleza", "ciudad", "aventura"], "alto", "Montañas Rocosas, lagos y
        » Toronto."),
45      "USA": d(["templado"], ["ciudad", "naturaleza", "aventura"], "alto", "Nueva York, parques na
     » cionales y rutas épicas."),
46      "MDV": d(["tropical"], ["playa"], "alto", "Bungalós sobre aguas cristalinas."),
47      }
```

templates/index.html

Página principal: formulario, buscador y zona de resultados

```
1       <!doctype html>
2       <html lang="es">
3       <head>
4       <meta charset="utf-8">
5       <meta name="viewport" content="width=device-width, initial-scale=1">
6       <title>¿Dónde viajarías?</title>
7       <link rel="preconnect" href="https://fonts.googleapis.com">
8       <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
9       <link
        href="https://fonts.googleapis.com/css2?family=Bricolage+Grotesque:wght@600;800&family=F
     » igtree:wght@400;600&display=swap" rel="stylesheet">
10      <link rel="stylesheet" href="/static/style.css">
11      </head>
12      <body>
13      <header class="portada">
14      <h1>¿Dónde viajarías?</h1>
15      <p>Contanos cómo imaginás tu próximo viaje y te sugerimos países que encajan.</p> 16
                </header>
 17
18          <main>
19          <form id="preferencias" class="ticket">
20          <fieldset class="ancho">
21          <legend><i>1</i>Tipo de viaje</legend>
22          <label><input type="radio" name="tipo" value="cualquiera" checked><span>Cualquiera</span
     » ></label>
23          <label><input type="radio" name="tipo" value="playa"><span>Playa</span></label>
24          <label><input type="radio" name="tipo" value="aventura"><span>Aventura</span></label>
25          <label><input type="radio" name="tipo" value="cultura"><span>Cultura</span></label>
26          <label><input type="radio" name="tipo" value="ciudad"><span>Ciudad</span></label>
```

```
27          <label><input type="radio" name="tipo" value="naturaleza"><span>Naturaleza</span></label
            » >
28          <label><input type="radio" name="tipo" value="gastronomia"><span>Gastronomía</span></lab
            » el>
29          </fieldset>
30          <fieldset>
31          <legend><i>2</i>Clima</legend>
32          <label><input type="radio" name="clima" value="cualquiera" checked><span>Cualquiera</spa
      » n></label>
33          <label><input type="radio" name="clima" value="tropical"><span>Tropical</span></label>
34          <label><input type="radio" name="clima" value="templado"><span>Templado</span></label>
35          <label><input type="radio" name="clima" value="seco"><span>Seco y soleado</span></label>
            36  <label><input type="radio" name="clima" value="frio"><span>Frío</span></label>
37          </fieldset>
38          <fieldset>
39          <legend><i>3</i>Presupuesto</legend>
40          <label><input type="radio" name="presupuesto" value="cualquiera" checked><span>Cualquier
            » a</span></label>
41          <label><input type="radio" name="presupuesto" value="bajo"><span>$ Económico</span></lab
            » el>
42          <label><input type="radio" name="presupuesto" value="medio"><span>$$ Moderado</span></la
            » bel>
43          <label><input type="radio" name="presupuesto" value="alto"><span>$$$ Alto</span></label>
44          </fieldset>
45          <div class="acciones ancho">
46          <button type="submit" class="principal">Ver destinos</button>
47          <small>Podés dejar cualquier opción en “Cualquiera”.</small>
48          </div>
49          </form>
 50
51            <section class="explorar">
52            <h2>¿Ya tenés algo en mente?</h2>
53            <div class="herramientas">
54            <label class="buscador">
55            <span>Buscar un país o una capital</span>
56            <input id="busqueda" type="search" placeholder="Japón, Lima, Portugal…" autocomplete="
      » off">
57            </label> 58          <button id="azar" type="button" class="secundario">Sorprendeme con
              un destino</button>
59        </div>
60        </section>
 61
62      <p id="estado" role="status" aria-live="polite"></p>
63      <section id="resultados" class="grilla"></section>
64      </main>
 65
66    <footer>Datos de <a href="https://restcountries.com">REST Countries</a>. Recomendaciones por r
      » eglas, sin inteligencia artificial.</footer>
67    <script src="/static/app.js"></script>
68    </body>
69    </html>
```

static/style.css

Estilos de la interfaz

```
1    :root{--tinta:#12263f;--cielo:#e8f1f8;--mar:#1b998b;--mar-os:#0b6159;--sol:#f2b134;-papel:#fff;
     » --linea:#d3e0ec;--suave:#4d6a86;
2    --titulo:"Bricolage Grotesque",system-ui,sans-serif;--texto:"Figtree",system-ui,sans-serif}
3    *{box-sizing:border-box}
4    body{margin:0;background:var(--cielo);color:var(--tinta);font:400 1rem/1.55 var(--texto)}
5    h1,h2,h3{font-family:var(--titulo);margin:0}
  6
7     .portada{position:relative;overflow:hidden;isolation:isolate;textalign:center;color:#fff;paddi
      n
     » g:5rem 1.25rem 8rem;
8     background:radial-gradient(ellipse at 15% 110%,#2a5d93 0,transparent 55%),radial-gradient(elli
     » pse at 90% -10%,#1b6f7a 0,transparent 45%),var(--tinta)}
9     .portada::before{content:"";position:absolute;inset:0;z-index:-
      1;background:radialgradient(rgba
     » (255,255,255,.16) 1.2px,transparent 1.5px) 0 0/24px 24px;
10    -webkit-mask-image:linear-gradient(#000,transparent 85%);mask-image:linear-gradient(#000,trans
     » parent 85%)}
11    .portada::after{content:"";position:absolute;inset:auto 0 0;height:70%;z-
      index:1;opacity:.75;ba     » ckground:
12    url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 800 200' preser
     » veAspectRatio='none'%3E%3Cpath d='M-10 170C200 20 420 230 810 40' fill='none' stroke='%23f2b134'
»  stroke-width='2.5' stroke-dasharray='3 11' stroke-linecap='round'/%3E%3C/svg%3E") center/100% 1
     » 00% no-repeat}
13    h1{font-size:clamp(2.8rem,9vw,5.6rem);font-weight:800;letter-spacing:-.035em;line-height:.98}
14    .portada p{max-width:34rem;margin:1.2rem auto 0;color:#c9d9ea;font-size:1.2rem}
15    main{max-width:70rem;margin:-4.5rem auto 3rem;padding:0 1.25rem}
 16
17    .ticket{background:var(--papel);border-radius:1.4rem;padding:2rem;display:grid;grid-
      templatecol
     » umns:1fr 1fr;gap:1.75rem 2rem;
18    box-shadow:0 2px 4px rgba(18,38,63,.06),0 28px 56px -24px rgba(18,38,63,.4)}
19    .ancho{grid-column:1/-1}
20    fieldset{border:0;margin:0;padding:0;display:flex;flex-wrap:wrap;gap:.5rem;align-
      content:flexst
     » art;min-width:0}
21    legend{display:flex;align-items:center;gap:.6rem;width:100%;float:left;font:600 1.15rem var(-
      ti
     » tulo);margin-bottom:.75rem;padding:0}
22    legend+*{clear:left}
23    legend i{display:grid;place-
      content:center;width:1.7rem;height:1.7rem;borderradius:50%;backgrou
     » nd:var(--tinta);color:#fff;font:700 .85rem var(--texto);font-style:normal}
24    .ticket label{position:relative}
25    .ticket input{position:absolute;opacity:0;inset:0;cursor:pointer;margin:0}
26    .ticket label span{display:block;padding:.55rem 1.1rem;border:1.5px solid var(--
      linea);borderra
     » dius:99px;transition:.15s;background:#fff;font-weight:500}
27    .ticket input:hover+span{border-color:var(--mar);background:#f1faf9}
28    .ticket input:checked+span{background:var(--tinta);border-color:var(--
      tinta);color:#fff;fontwei
     » ght:600;box-shadow:0 5px 12px -5px rgba(18,38,63,.55)}
29    .ticket input:focus-visible+span,button:focus-visible,.buscador input:focusvisible{outline:3px
     » solid var(--sol);outline-offset:2px}  30  .acciones{display:flex;flex-
wrap:wrap;alignitems:center;gap:1rem;padding-top:1.5rem;border-top:
     » 2px dashed var(--linea)}
31  .acciones small{color:var(--suave);font-size:.92rem}
32  button{font:600 1.02rem var(--texto);border-radius:.75rem;padding:.9rem 1.6rem;cursor:pointer;bo
     » rder:2px solid var(--tinta);transition:.15s}
33  .principal{background:var(--sol);color:var(--tinta);box-shadow:0 5px 0 var(--tinta)}
34  .principal:hover{transform:translateY(2px);box-shadow:0 3px 0 var(--tinta)}
35  .secundario{background:#fff;color:var(--tinta)} 36  .secundario:hover{background:var(-
    tinta);color:#fff}
 37
38  .explorar{margin-top:2.5rem}
39  .explorar h2{font-size:1.6rem;margin-bottom:.9rem}
40  .herramientas{display:flex;flex-wrap:wrap;gap:1rem;align-items:end}
```

```
41  .buscador{flex:1 1 18rem;display:grid;gap:.35rem;font-weight:600}
42  .buscador input{font:inherit;font-weight:400;padding:.85rem 1rem .85rem 2.9rem;border:2px solid
     » var(--linea);border-radius:.75rem;background:#fff
```

```
43  url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 24 24' fill='no
    » ne' stroke='%234d6a86' stroke-width='2' stroke-linecap='round'%3E%3Ccircle cx='11' cy='11' r='7'
     » /%3E%3Cpath d='M20 20l-3.5-3.5'/%3E%3C/svg%3E") 1rem center/1.2rem no-repeat}
44  .buscador input:focus{border-color:var(--mar)}
45  #estado{min-height:1.6rem;font:600 1.25rem var(--titulo);margin:1.75rem 0 .9rem}
 46
47  .grilla{display:grid;grid-template-columns:repeat(auto-fill,minmax(18.5rem,1fr));gap:1.5rem}
48  .pase{position:relative;background:var(--papel);border:1px solid var(--linea);border-radius:1.25
     » rem;overflow:hidden;display:flex;flex-direction:column;  49    box-shadow:0 1px 2px
rgba(18,38,63,.06),0 16px 30px -18px rgba(18,38,63,.35);animation:entra .      » 4s
both;animationdelay:calc(var(--i,0)*70ms)}
50  @keyframes entra{from{opacity:0;transform:translateY(10px)}}
51  .bandera{position:relative;height:11.5rem;overflow:hidden;isolation:isolate;border-bottom:2px da
    » shed var(--linea)}
52  .bandera::before{content:"";position:absolute;inset:-24px;z-index:-1;background:var(--bg) center
    » /cover;filter:blur(18px) saturate(1.3)}
53  .bandera img{width:100%;height:100%;object-fit:contain;padding:1.3rem;display:block;filter:drop-
    » shadow(0 6px 10px rgba(0,0,0,.3))}
54  .sello{position:absolute;top:9.2rem;right:1rem;width:4.6rem;height:4.6rem;border-radius:50%;back
     » ground:var(--sol);border:3px double var(--tinta);  55    display:grid;placecontent:center;text-
align:center;font:800 1.2rem/1 var(--titulo);transform:
     » rotate(-9deg);box-shadow:0 4px 10px rgba(18,38,63,.25)}
56  .sello small{font:600 .6rem var(--texto)}
57  .cuerpo{padding:1.3rem 1.3rem 1.5rem;display:flex;flex-direction:column;gap:.6rem;flex:1}
58  h3{font-size:1.75rem;line-height:1.08;padding-right:4.4rem}
59  .region{margin:0;color:var(--suave);font-size:.92rem}
60  .nota{margin:.2rem 0 0;padding-left:.8rem;border-left:3px solid var(--sol);font-weight:600}
61  .razones{margin:0;color:var(--mar-os);font-weight:600;font-size:.9rem}
62  .razones::before{content:"";display:inline-block;width:.55rem;height:.3rem;margin:0 .45rem .2rem
     »  0;border:solid var(--mar);border-width:0 0 2.5px 2.5px;transform:rotate(-45deg)}
63  .datos{display:grid;grid-template-columns:1fr 1fr;gap:.6rem;margin-top:.4rem}
64  .dato{background:var(--cielo);border-radius:.85rem;padding:.65rem .8rem;min-width:0}
65  .dato.ancho{grid-column:1/-1}
66  .dato b{display:flex;align-items:center;gap:.4rem;font-size:.8rem;font-weight:600;color:var(--su
     » ave);margin-bottom:.1rem}
67  .dato svg{color:var(--mar);flex:none}
68  .dato span{display:block;font-weight:600;font-size:.95rem;overflow-wrap:anywhere}
69  .dato .vacio{color:var(--suave);font-weight:400}
70  .tags{list-style:none;padding:.8rem 0 0;margin:auto 0 0;display:flex;flex-wrap:wrap;gap:.4rem}
71  .tags li{border-radius:99px;padding:.18rem .75rem;font-size:.82rem;font-weight:600}
72  .tipo{background:#d6f1ed;color:var(--mar-os)}.clima{background:#fdecc4;color:#734d00}.pres{backg
     » round:#dde6f2;color:var(--tinta)}
 73
74  .esqueleto{height:27rem;background:linear-gradient(100deg,#fff 30%,#eef4f9 50%,#fff 70%) 0 0/200
     » % 100%;animation:brillo 1.2s linear infinite}
75  @keyframes brillo{to{background-position:-200% 0}}
76  footer{background:var(--tinta);color:#a9bdd2;text-align:center;padding:2rem 1rem;font-size:.92re
     » m}
77  footer a{color:#fff}
78  @media (max-width:42rem){.ticket{grid-template-columns:1fr;padding:1.4rem}.portada{padding:3.5re
     » m 1.25rem 7rem}}
79  @media (prefers-reduced-motion:reduce){*{transition:none!important;animation:none!important}}
```

static/app.js

Lógica del navegador: formulario, búsqueda, destino aleatorio y tarjetas

```
1    const $ = (s) => document.querySelector(s);
2    const estado = $("#estado"), resultados = $("#resultados");
3    const esc = (s) => String(s).replace(/[&<>"']/g, (c) =>
4    ({ "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;", "'": "&#39;" }[c]));
  5
6     const PRES = { bajo: "Económico", medio: "Moderado", alto: "Alto" };
7     const NIVEL = { bajo: 1, medio: 2, alto: 3 };
8     const ICONOS = {
9     capital: '<path d="M12 21s-7-6.2-7-11a7 7 0 1 1 14 0c0 4.8-7 11-7 11z"/><circle cx="12" cy="10
      » " r="2.5"/>',
10    poblacion: '<circle cx="9" cy="8" r="3.2"/><path d="M3 20c0-3.3 2.7-6 6-6s6 2.7 6 6M16 5.2a3.2
     »  3.2 0 0 1 0 6M21 20c0-2.6-1.500-4.800-3.600-5.600"/>',
11    idiomas: '<path d="M4 5h16v11h-9l-4 4v-4H4z"/>', 12    moneda: '<circle cx="12" cy="12"
      r="8.500"/><path d="M14.800 9.200c-.6-.8-1.600-1.200-2.800-1.
     » 200-1.600 0-2.800.8-2.800 2s1.200 1.700 2.800 2 2.800.8 2.800 2-1.200 2-2.800 2c-1.200 0-2.200-.
» 4-2.800-1.200M12 6.500V8m0 8v1.500"/>',
```

```
13    };
14    const ico = (k) => `<svg viewBox="0 0 24 24" width="16" height="16" fill="none" stroke="currentC
      » olor" stroke-width="1.8" stroke-linecap="round" stroke-linejoin="round" aria-
      hidden="true">${ICO
      » NOS[k]}</svg>`;
15    const dato = (icono, titulo, valores, ancho) => `<div class="dato${ancho ? " ancho" : ""}"><b>${
      » ico(icono)}${titulo}</b>` +
16    (valores.length ? `<span>${esc(valores.join(", "))}</span>` : `<span class="vacio">Sin datos</
      » span>`) + `</div>`;
17
18          function tarjeta(p, n) {
19          const i = p.info;
20          const li = (clase, t) => `<li class="${clase}">${esc(t)}</li>`;
21          const tags = i ? [...i.tipos.map((t) => li("tipo", t)), ...i.clima.map((t) => li("clima",
            t)),
22          li("pres", "$".repeat(NIVEL[i.presupuesto]) + " " + PRES[i.presupuesto])].join("") : "";
23          return `<article class="pase" style="--i:${n}">
24          <div class="bandera" style="--bg:url('${esc(encodeURI(p.bandera))}')">
25          <img src="${esc(p.bandera)}" alt="Bandera de ${esc(p.nombre)}" loading="lazy">
26          </div>
27          ${p.puntaje != null ? `<span class="sello">${p.puntaje}%<small>afinidad</small></span>`
            : ""
      » }
28          <div class="cuerpo">
29          <h3>${esc(p.nombre)}</h3>
30          <p class="region">${esc(p.region)}</p>
31          ${i ? `<p class="nota">${esc(i.nota)}</p>` : ""}
32          ${p.razones && p.razones.length ? `<p class="razones">${p.razones.map(esc).join(",
            ")}</p>
      » ` : ""}
33          <div class="datos">
34          ${dato("capital", "Capital", [p.capital])}
35          ${dato("poblacion", "Población", [p.poblacion.toLocaleString("es-AR")])}
36          ${dato("idiomas", "Idiomas", p.idiomas, true)}
37          ${dato("moneda", "Moneda", p.monedas, true)}
38          </div>
39          ${tags ? `<ul class="tags">${tags}</ul>` : ""}
40          </div>
41          </article>`;
42          }
43
44      async function pedir(url, titulo, vacio) {
45      estado.textContent = "Buscando destinos…";
46      resultados.innerHTML = '<div class="pase esqueleto"></div>'.repeat(3);
47      try {
48      const r = await fetch(url);
49      const data = await r.json().catch(() => ({}));
50      if (!r.ok) throw new Error(data.error || `El servidor respondió con error ${r.status}.`);
51      resultados.innerHTML = data.paises.map((p, n) => tarjeta(p, n)).join("");
52      estado.textContent = data.paises.length ? titulo : vacio;
53      } catch (e) {
54      resultados.innerHTML = "";
55      estado.textContent = e.message + "";
56      }
57      }
58
59      $("#preferencias").addEventListener("submit", (e) => { 60
e.preventDefault();
61      const q = new URLSearchParams(new FormData(e.target));
62      pedir("/api/recomendar?" + q, "Destinos que encajan con tu viaje:",
63      "No hay destinos con esa combinación. Probá dejar alguna opción en “Cualquiera”.");
64      });
65
66      let espera;
```

```
67      $("#busqueda").addEventListener("input", (e) => {
68      clearTimeout(espera);
69      const q = e.target.value.trim();
70      if (q.length < 2) { resultados.innerHTML = ""; estado.textContent = ""; return; }
71      espera = setTimeout(() => pedir("/api/buscar?q=" + encodeURIComponent(q),
72      "Resultados para “" + q + "”:", "No encontramos “" + q + "”. Revisá la ortografía."), 300);
73      });
 74
75    $("#azar").addEventListener("click", () =>
76    pedir("/api/aleatorio", "Tu destino sorpresa:", "No pudimos elegir un destino."));
```


requirements.txt

Dependencias de Python

```
1  flask>=3.0
2  requests>=2.31
```